

1. Objetivo

Utilizar los hilos de ejecución de un *kernel* lanzado con dos dimensiones.

2. Ejercicio

Dada una matriz rectangular con *F* filas y *C* columnas inicializada en el *host* con números aleatorios entre 1 y 9, lanzar un *kernel* que genere una nueva matriz donde el valor de las columnas pares se mantenga y sea 0 en el resto.

El *kernel* debe lanzarse como un solo bloque bidimensional donde cada hilo de ejecución se encargue de obtener un único elemento de la matriz final. El programa debe mostrar las propiedades del dispositivo, el número de hilos lanzados en cada eje, la matriz inicial y la matriz final.

3. Solución propuesta

```
/*
* ARQUITECTURA DE COMPUTADORES
* 2º Grado en Ingenieria Informatica
* Introduccion a la programacion en CUDA
*
* Basico 4: "Arrays multidimensionales"
* >> Mantener columnas pares
*
*/
////////////////////////////////////////////////////////////////////////
// includes
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <cuda_runtime.h>

////////////////////////////////////////////////////////////////////////
// defines
#define COLUMNAS 21 // Numero de columnas -> eje x
#define FILAS 6     // Numero de filas    -> eje y

// declaracion de funciones
// HOST: funcion llamada desde el host y ejecutada en el host
__host__ void propiedades_Device();
// Funcion que imprime las propiedades del device
```

```
// GLOBAL: funcion llamada desde el host y ejecutada en el device (kernel)
__global__ void MatFinal(int *entrada, int *salida)
// Función que mantiene las columnas pares
{
	// KERNEL BIDIMENSIONAL DE UN SOLO BLOQUE: (X,Y)
	// indice de columna: EJE x
	int columna = threadIdx.x;
	// indice de fila: EJE y
	int fila = threadIdx.y;
	// indice lineal
	int globalID = columna + fila * COLUMNAS;
	// Mantenemos las columnas pares
	if (columna % 2 == 0)
	{
		salida[globalID] = entrada[globalID];
	}
	else
	{
		salida[globalID] = 0;
	}
}

////////////////////////////////////////////////////////////////////////
// MAIN: rutina principal ejecutada en el host
int main(int argc, char** argv)
{
	// buscando dispositivos
	int deviceCount;
	cudaGetDeviceCount(&deviceCount);
	if (deviceCount == 0)
	{
		printf("!!!!!No se han encontrado dispositivos CUDA!!!!!\n");
		printf("<pulsa [INTRO] para finalizar>");
		getchar();
		return 1;
	}
	else
	{
		propiedades_Device();
	}

	// declaraciones
	int *hst_Entrada, *hst_Salida;
	int *dev_Entrada, *dev_Salida;

	// reserva en el host
	hst_Entrada = (int*)malloc(FILAS*COLUMNAS * sizeof(int));
	hst_Salida = (int*)malloc(FILAS*COLUMNAS * sizeof(int));

	// reserva en el device
	cudaMalloc((void**)&dev_Entrada, FILAS*COLUMNAS * sizeof(int));
	cudaMalloc((void**)&dev_Salida, FILAS*COLUMNAS * sizeof(int));

	// incializacion
	for (int i = 0; i<FILAS*COLUMNAS; i++)
	{
		hst_Entrada[i] = 1 + rand() % 9; // numeros aleatorios entre 1 y 9
		hst_Salida[i] = 0;
	}

	// dimensiones del kernel
	// 1 Bloque
	dim3 Nbloques(1);
	// bloque bidimensional (x,y)
	// Eje x-> COLUMNAS
	// Eje y-> FILAS
	dim3 hilosB(COLUMNAS, FILAS);

	// copia de datos hacia el device
	cudaMemcpy(dev_Entrada, hst_Entrada, FILAS*COLUMNAS * sizeof(int),
cudaMemcpyHostToDevice);

	// Numero de hilos
	printf("> KERNEL de 1 BLOQUE con %d HILOS:\n", COLUMNAS*FILAS);
	printf("  eje x -> %2d hilos\n  eje y -> %2d hilos\n", COLUMNAS, FILAS);

	// llamada al kernel
	MatFinal << <Nbloques, hilosB >> >(dev_Entrada, dev_Salida);

	// recogida de datos desde el device
```

```
	cudaMemcpy(hst_Salida, dev_Salida, FILAS*COLUMNAS * sizeof(int),
cudaMemcpyDeviceToHost);

	// impresion de resultados
	printf("> MATRIZ ORIGINAL:\n");
	for (int i = 0; i<FILAS; i++)
	{
		for (int j = 0; j<COLUMNAS; j++)
		{
			printf("%2d ", hst_Entrada[j + i*COLUMNAS]);
		}
		printf("\n");
	}
	printf("\n");
	printf("> MATRIZ FINAL:\n");
	for (int i = 0; i<FILAS; i++)
	{
		for (int j = 0; j<COLUMNAS; j++)
		{
			printf("%2d ", hst_Salida[j + i*COLUMNAS]);
		}
		printf("\n");
	}

	// salida del programa
	time_t fecha;
	time(&fecha);
	printf("*************************************************\n");
	printf("Programa ejecutado el: %s\n", ctime(&fecha));
	printf("<pulsa [INTRO] para finalizar>");
	getchar();
	return 0;
}
////////////////////////////////////////////////////////////////////////
// HOST: funcion llamada desde el host y ejecutada en el host
__host__ void propiedades_Device()
{
	int deviceID;
	cudaGetDevice(&deviceID);
	int runtimeVersion;
	cudaRuntimeGetVersion(&runtimeVersion);
	cudaDeviceProp deviceProp;
	cudaGetDeviceProperties(&deviceProp, deviceID);
	// calculo del numero de cores (SP)
	int cudaCores = 0;
	int SM = deviceProp.multiProcessorCount;
	int major = deviceProp.major;
	int minor = deviceProp.minor;
	const char *archName;
	switch (major)
	{
	case 1:
		//TESLA
		archName = "TESLA";
		cudaCores = 8;
		break;
	case 2:
		//FERMI
		archName = "FERMI";
		if (minor == 0)
			cudaCores = 32;
		else
			cudaCores = 48;
		break;
	case 3:
		//KEPLER
		archName = "KEPLER";
		cudaCores = 192;
		break;
	case 5:
		//MAXWELL
		archName = "MAXWELL";
		cudaCores = 128;
		break;
	case 6:
		//PASCAL
		archName = "PASCAL";
		cudaCores = 64;
		break;
	case 7:
		//VOLTA (7.0) TURING (7.5)
```

```
                cudaCores = 64;
                if (minor == 0)
                        archName = "VOLTA";
                else
                        archName = "TURING";
                break;
        case 8:
                //AMPERE
                archName = "AMPERE";
                cudaCores = 64;
                break;
        default:
                //ARQUITECTURA DESCONOCIDA
                archName = "DESCONOCIDA";
                cudaCores = 0;
        }
        // presentacion de propiedades
        printf("*************************************************\n");
        printf("DEVICE %d: %s\n", deviceID, deviceProp.name);
        printf("*************************************************\n");
        printf("> CUDA Toolkit                      \t: %d.%d\n", runtimeVersion / 1000,
(runtimeVersion % 1000) / 10);
        printf("> Capacidad de Computo              \t: %d.%d\n", major, minor);
        printf("> Arquitectura CUDA                 \t: %s\n", archName);
        printf("> No. de MultiProcesadores          \t: %d\n", SM);
        printf("> No. de CUDA Cores (%dx%d)         \t: %d\n", cudaCores, SM, cudaCores*SM);
        printf("> No. maximo de Hilos (por bloque)\t: %d\n", deviceProp.maxThreadsPerBlock);
        printf("> Memoria Global (total)            \t: %zu MiB\n", deviceProp.totalGlobalMem /
(1024 * 1024));
        printf("*************************************************\n");
}
```